

Agent Instructions: Rebuild the OpenStates Jurisdictions Data Pipeline

Objective

Design and implement a reproducible pipeline for the `openstates/jurisdictions` repository that creates a comprehensive, canonical representation of U.S. government entities and the geopolitical divisions over which they exercise jurisdiction.

The resulting repository should contain fully populated `Jurisdiction.yaml` and `Division.yaml` records.

These YAML files are the durable source of truth.

Downstream systems—including graph databases, relational databases, APIs, and search indexes—must be reproducible from the YAML repository.

The target conceptual model is:

```
Government Entity
  |
  | exercises jurisdiction over
  v
Geopolitical Division
  |
  | identified geographically by
  v
Census GEOID / other authoritative identifier
  |
  | geometry available from
  v
TIGERweb / authoritative geometry provider
```

Open Civic Data Division IDs remain the canonical human-readable, hierarchical identifier for divisions.

1. Core principles

Follow these principles throughout the implementation.

Census identifies governments

Use the Census Government Units Survey / Government Organization data as the primary authoritative source for the universe of U.S. state and local governments.

Census Government data answers:

```
Does this government exist?  
What is its official name?  
What type of government is it?  
What Census government-unit identifier identifies it?  
Where is it administratively located?
```

Census TIGER identifies geographic divisions

Use Census TIGER/TIGERweb as the primary source for Census-recognized geopolitical geography.

TIGER answers:

```
What geographic division exists?  
What is its Census GEOID?  
What kind of geography is it?  
What is its authoritative boundary?
```

Do not require users or pipeline developers to run TIGER locally in PostGIS.

Where possible, Division records should contain a URL that directly returns GeoJSON from TIGERweb.

Open Civic Data IDs provide civic hierarchy

Preserve Open Civic Data Division Identifiers because they provide:

- human-readable IDs;
- meaningful administrative hierarchy;
- predictable nested paths;
- recursive lookup semantics;
- interoperability with existing OpenStates/Open Civic Data projects.

For example:

```
ocd-division/country:us/state:ca/county:alameda/place:berkeley
```

Census GEOIDs do NOT replace OCD Division IDs.

The identifiers solve different problems:

```
OCD Division ID
  human-readable civic identity + hierarchy

Census GEOID
  Census geographic identity

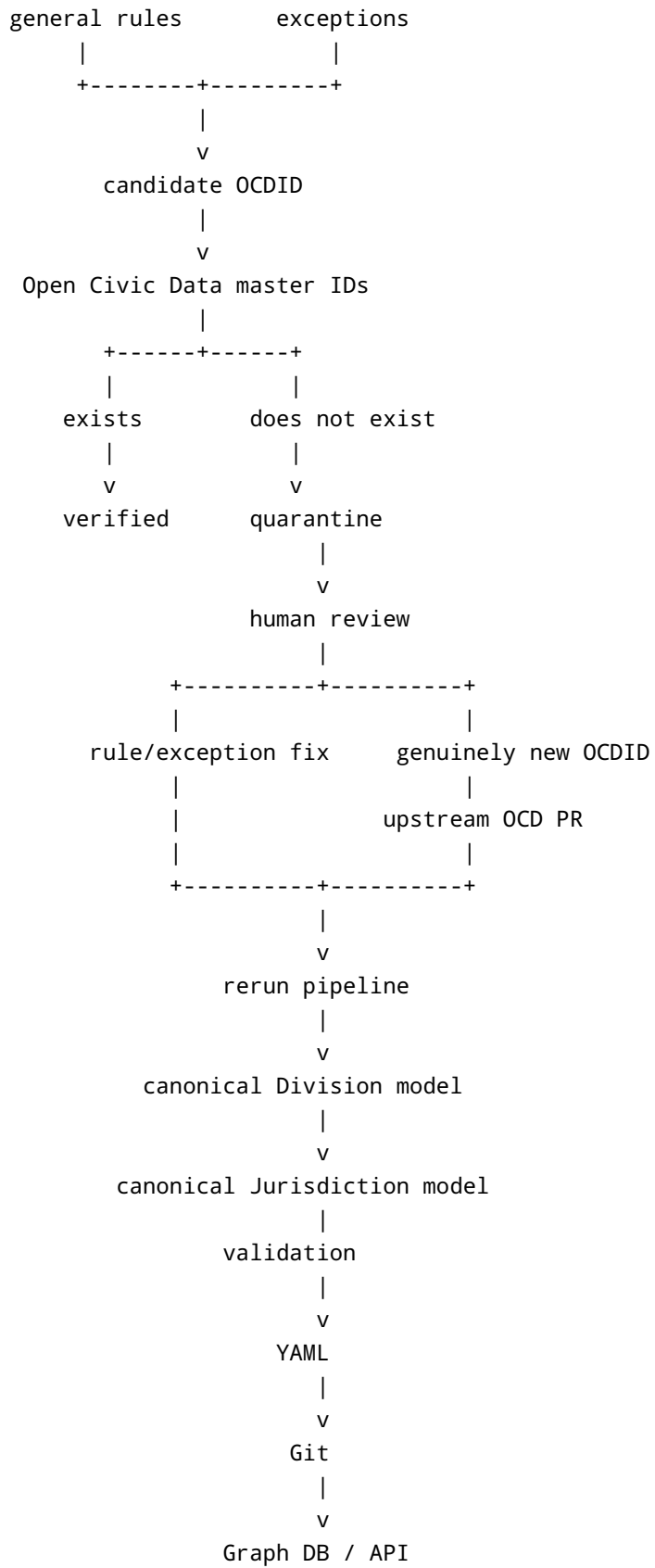
Census Government ID
  government-entity identity

Geometry URL
  authoritative boundary retrieval
```

2. Overall pipeline

Implement approximately the following flow:

```
Census Government Organization / GUS
|
v
normalize governments
|
v
classify government type
|
v
expected Division type
|
v
Census TIGER data
|
v
resolve geographic entity
|
v
candidate Division facts
|
v
OCDID rule engine
|
+-----+-----+
|               |
```



Generation must be deterministic wherever authoritative source data makes that possible.

3. Source ingestion

Source files should be downloaded once per source release and cached.

Do not repeatedly call Census APIs while testing the pipeline.

Prefer bulk datasets when available.

Suggested structure:

```
data/  
  raw/  
    census/  
      government_units/  
        2025/  
      tiger/  
        2025/  
    ocd/  
      master/  
  cache/
```

Intermediate normalized datasets may use Parquet.

DuckDB may query these files but should not be required as persistent pipeline state.

For example:

```
cache/  
  census_governments_2025.parquet  
  census_states_2025.parquet  
  census_counties_2025.parquet  
  census_places_2025.parquet  
  census_cousub_2025.parquet  
  census_school_districts_2025.parquet
```

Pipeline tests should therefore run repeatedly without network access once sources have been downloaded.

4. Normalize Census government entities

Convert raw Census Government Organization records into a clean internal representation.

Conceptually:

```
class GovernmentRecord:
    census_government_id: str

    name: str
    normalized_name: str

    government_type: str
    government_subtype: str | None

    state_fips: str
    county_fips: str | None

    state: str
    county: str | None
    place_name: str | None

    website: str | None

    source: Source
```

Do not generate YAML at this stage.

Preserve raw authoritative names as well as normalized forms.

Normalization exists for matching, not to overwrite authoritative names.

5. Government-to-Division classification

Create deterministic rules translating Census government types into expected geographic Division types.

Examples:

```
State government
→ state

County government
```

→ county

Municipal government

→ incorporated place / appropriate place geography

Town/township government

→ county subdivision / MCD where applicable

School district

→ applicable TIGER school district layer

Special district

→ special handling

Keep this step independent from actual geographic matching.

It answers:

What TYPE of division should this government govern?

not:

Which exact division is it?

6. TIGER geography resolution

Resolve normal government types to Census/TIGER geographic records.

Use direct identifier joins whenever possible.

Avoid fuzzy matching when deterministic Census identifiers are available.

Target high-confidence mappings for:

- states;
- counties;
- incorporated places;
- functioning county subdivisions / MCDs;
- elementary school districts;
- secondary school districts;
- unified school districts;

- other Census-supported government geography.

Create an internal object conceptually like:

```
class CensusDivisionRecord:
    geography_type: str

    geoid: str
    geoidfq: str | None

    name: str
    namelsad: str | None

    state_fips: str
    county_fips: str | None
    place_fips: str | None
    cousub_fips: str | None

    geometry_source: Source
    geometry_url: str
```

Do not download and store polygons in Git.

7. Geometry strategy

The repository should contain authoritative references to geometry rather than thousands of geometry blobs.

For TIGER-supported Divisions, generate a TIGERweb URL that returns GeoJSON.

Conceptually:

```
GET Division.geometry.url
    ↓
GeoJSON FeatureCollection
```

No PostGIS requirement.

No local TIGER server requirement.

No ArcGIS request should be required during normal YAML generation if the URL can be constructed deterministically from service, layer and GEOID.

Geometry provider references must be provider-neutral at the model level.

Do not make a field such as `arcGIS_address` the generic geometry abstraction.

Prefer something conceptually like:

```
geometry:
  provider: census_tigerweb
  identifier: "0606000"
  url: https://...
  source: ...
```

The underlying service may happen to use ArcGIS REST, but the domain model should not depend upon ArcGIS.

8. OCD Division IDs

OCD Division IDs remain first-class canonical identifiers.

Do NOT replace them with Census GEOIDs.

Prefer readable identifiers such as:

```
ocd-division/country:us/state:ca/county:alameda/place:berkeley
```

rather than:

```
.../place:0606000
```

Store the GEOID separately.

The OCD hierarchy is a civic hierarchy, while the GEOID is an external geographic identifier.

9. Reuse existing Open Civic Data behavior

Before implementing new rules, inspect and reuse:

- `opencivicdata/ocd-division-ids`;
- OCDEP 2;

- existing OpenStates OCDID parsing/generation utilities;
- existing OpenStates exception logic;
- existing identifier normalization behavior;
- the canonical U.S. OCDID corpus.

Existing OCDIDs must remain stable.

Never regenerate an existing canonical OCDID merely because a new rule would produce a different path.

Use:

```
Existing canonical OCDID?
|
+-- YES → preserve it
|
+-- NO → generate candidate
```

10. OCDID rule engine

Implement candidate OCDID construction as an explicit rule engine.

Do not scatter hierarchy rules throughout ingestion or resolution code.

Conceptually:

```
candidate = rule_engine.generate(
    geography_type=division.type,
    ancestors=division.ancestors,
    names=division.names,
)
```

Return provenance along with the candidate:

```
OCDIDCandidate(
    value="ocd-division/...",
    rule="municipality.default",
    rule_version="...",
    transformations=[...],
)
```

Rules should define:

- hierarchy components;
- OCD segment types;
- slug normalization;
- required ancestors;
- treatment of special government types.

Where practical, rules should be declarative rather than large nested conditionals.

11. Exception handling

Exception handling must be separate from general rules.

Do NOT deform normal rules to accommodate exceptional governmental structures.

Examples likely requiring explicit rules include:

- independent cities;
- Alaska boroughs and equivalents;
- Louisiana parishes;
- consolidated city-counties;
- New England government structures;
- unusual county-equivalents;
- historical OCD naming conventions;
- legacy identifiers that must remain stable.

Support distinct exception categories such as:

```
identifier override
hierarchy override
slug/name override
geography mapping override
```

Human review should generally result in a new reusable rule or exception rather than manually editing generated YAML.

12. Validate against the Open Civic Data master repository

After generating a candidate OCDID, perform an exact lookup against the canonical Open Civic Data master repository.

The primary decision is binary:

```
if candidate_ocdid in canonical_ocdids:
    VERIFIED
else:
    QUARANTINED
```

Do not automatically canonicalize based on fuzzy similarity.

Fuzzy matching may be used only to assist review.

For example, quarantine output can show:

```
candidate:
  ocd-division/.../candidate

nearest canonical IDs:
  0.97 ...
  0.91 ...
```

but these suggestions must not silently alter identity.

13. Quarantine is a first-class pipeline stage

Unknown OCDIDs are not pipeline errors.

They are review records.

Maintain a structured quarantine dataset rather than producing only logs.

For every quarantined item include enough information to resolve it without rerunning discovery:

```

government:
  census_id: ...
  name: ...
  type: ...

division:
  census_geoid: ...
  name: ...
  type: ...

candidate:
  ocdid: ...
  rule: ...
  transformations: ...

nearest_master_ids:
  - ...

review:
  status: pending
  decision: null
  canonical_ocdid: null
  notes: null

```

Possible review outcomes:

```

rule bug
existing OCDID found through alternate mapping
known exception
new legitimate OCDID
government has no corresponding Division
bad source record

```

14. New legitimate OCDIDs

Do not let this repository silently establish a competing OCDID namespace.

When Census identifies a legitimate Division that does not exist in the canonical OCD repository:

```

candidate
↓
quarantine

```

```
↓  
human confirms new Division  
↓  
PR to canonical Open Civic Data Division ID repository  
↓  
canonical ID accepted  
↓  
refresh master  
↓  
rerun  
↓  
verified
```

This keeps OpenStates and Open Civic Data synchronized.

15. Jurisdiction vs Division semantics

Maintain a strict conceptual distinction.

Division

A Division is a geopolitical/geographic area.

Examples:

```
California  
Alameda County  
Berkeley  
a school district  
a legislative district
```

Jurisdiction

A Jurisdiction is a government entity exercising authority over one or more Divisions.

Examples:

```
State of California  
County of Alameda
```

City of Berkeley
Berkeley Unified School District

Do not conflate:

where a government office is located

with:

where that government has jurisdiction

16. Relationship direction

Prefer the conceptual relationship:

Jurisdiction --GOVERNS--> Division

rather than encoding a single `jurisdiction_id` as an intrinsic property of Division.

A Division may interact with more than one governmental entity.

A Jurisdiction may also govern or serve more than one Division.

Use an explicit relationship model where necessary.

Conceptually:

```
division_relationships:  
  - division_id: ...  
    relationship: governs
```

or:

```
Jurisdiction  
  GOVERNS  
Division
```

For regional and special-purpose governments, relationships may include:

GOVERNS
SERVES
OVERLAPS
CONTAINED_BY

Do not force all real-world authority into a false containment hierarchy.

17. Special districts

Special districts must be treated separately from ordinary TIGER-supported governments.

Examples include:

- water districts;
- transit authorities;
- fire districts;
- port authorities;
- mosquito abatement districts;
- utility districts;
- regional authorities.

A Census government-unit record does not imply that TIGER contains a matching polygon.

Do not manufacture Census GEOIDs.

It is acceptable to create a valid Jurisdiction before an authoritative Division geometry is available.

Potential sources for special-district boundaries may include:

- state GIS portals;
- district GIS services;
- authoritative ArcGIS FeatureServers;
- statutory descriptions;
- other official government sources.

These may later be related to Census Divisions via:

OVERLAPS county
SERVES county
OVERLAPS place

18. Data model: stable identity

Entity identity must remain stable across routine updates.

Do not generate UUIDs from `last_updated`, source release, geometry vintage or other mutable fields.

Prefer:

UUID5(OCDID)

for permanent identity.

Changing a:

- website;
- Census source release;
- geometry;
- boundary;
- name;
- source;
- retrieved timestamp

must not automatically create a new entity identity.

19. Temporality

Treat temporal data as first-class.

Distinguish at least:

```
valid time
    When was this fact true in the real world?

observation/source time
    When did our pipeline learn or verify it?
```

Examples of real-world temporal changes include:

- redistricting;
- annexation;
- incorporation;
- government dissolution;

- school district consolidation;
- government renaming;
- changed websites;
- jurisdictional boundary changes.

Do not rely on Git history as the temporal data model.

Git tells us:

```
when the repository changed
```

not necessarily:

```
when the government or boundary changed in reality
```

Represent meaningful validity explicitly.

20. Geometry temporality

A Division identity can persist while its geometry changes.

For example, a legislative district OCDID might remain:

```
ocd-division/country:us/state:ca/sld1:14
```

while successive boundary definitions are stored.

Conceptually:

```
geometries:  
  
- valid_from: 2012-01-01  
  valid_to: 2022-12-04  
  source: ...  
  
- valid_from: 2022-12-05  
  valid_to: null  
  source: ...
```

The Division UUID remains stable.

This allows consumers to ask:

What is this Division now?
What did it represent in 2018?
Which boundary applied during a specific election?

21. Source objects and "vintage"

The project already has a Source/Sourcing object.

Reuse it for dataset release/vintage metadata rather than adding redundant `vintage` properties everywhere.

A source should be capable of expressing concepts such as:

```
source:
  name: Census TIGER/Line
  dataset: TIGERweb
  release: "2025"
  published_at: ...
  retrieved_at: ...
  url: ...
```

The exact existing Source schema should be extended only if necessary.

Do not confuse source release with real-world validity.

For example:

```
valid_from
  when the boundary became effective

source.release
  which TIGER release describes it

retrieved_at
  when our pipeline downloaded or observed it
```

All three can differ.

Use Source to capture provenance and source-specific concepts such as:

- vintage;
- release;
- edition;
- plan;
- version.

22. Division identifiers

Avoid making the core Division schema excessively Census-specific.

Existing fields such as:

```
statefp
countyfp
placefp
cousubfp
sldust
sldlst
geoid
geoid_12
geoid_14
```

should conceptually be external identifiers/properties associated with Census.

Prefer a structured external-identifier representation.

For example:

```
identifiers:
  census:
    geoid: "0606000"
    statefp: "06"
    placefp: "06000"
    source: ...
```

or an equivalent model consistent with the existing project conventions.

Keep OCDID separate and first-class.

23. Names

Separate legal names, common/display names, and historical aliases where useful.

Conceptually:

```
name: City of Berkeley
display_name: Berkeley

names:
  - name: City of Berkeley
    type: legal
    valid_from: ...

  - name: Berkeley
    type: common
```

Do not overwrite historical identity merely because the current government name changes.

24. URLs

Government websites are enrichment data, not prerequisites for entity existence.

A government without a confidently known website should still be generated.

Do not fail or quarantine an otherwise valid government solely because:

```
url = unknown
```

Prefer temporal/source-aware URL records if supported by the existing schema.

Conceptually:

```
urls:
  - type: official
    url: https://...
    valid_from: ...
    valid_to: null
    source: ...
```

Website resolution can be performed as a separate enrichment pipeline.

Potential source hierarchy:

```
Census-provided URL
  ↓
.gov registry
  ↓
local candidate matching
  ↓
LLM/reranker
  ↓
live search fallback
```

Never ask an LLM to invent an official government URL.

Use an LLM only to verify/rerank known candidates.

25. Jurisdiction scope

Keep the core Jurisdiction model focused on independent governmental entities.

Do not assume every organization operating within a government is a Jurisdiction.

Conceptually distinguish:

```
City of Berkeley
  Jurisdiction / government

Berkeley City Council
  organization within government

Mayor
  office

Planning Commission
  organization/board
```

The current project may retain broader classifications for compatibility, but new architecture should avoid conflating these conceptual levels.

Optional governance-specific information such as legislative sessions should not be mandatory for every government type.

26. Graph database

The graph database is downstream.

The YAML repository remains canonical.

The graph should be disposable and fully reconstructable.

Potential graph nodes:

```
Jurisdiction
Division
ExternalIdentifier
Source
GeometryVersion
```

Potential graph edges:

```
GOVERNS
PARENT_OF
SERVES
OVERLAPS
CONTAINED_BY
IDENTIFIED_BY
SOURCED_FROM
HAS_GEOMETRY
```

OCDID path hierarchy can generate many `PARENT_OF` relationships automatically.

Example:

```
United States
↓
California
↓
Alameda County
```

↓
Berkeley

The graph can then efficiently answer recursive questions while preserving human-readable OCD paths.

27. Pipeline status

Every processed government should terminate in an explicit status.

Suggested statuses:

```
COMPLETE  
  
NO_GEOGRAPHY  
  
NEW_OCDID  
  
AMBIGUOUS_OCDID  
  
DIVISION_NOT_FOUND  
  
SOURCE_ERROR
```

Generate summary statistics from every run.

For example:

Government units processed	N
COMPLETE	N
NO_GEOGRAPHY	N
NEW_OCDID	N
AMBIGUOUS_OCDID	N
DIVISION_NOT_FOUND	N
SOURCE_ERROR	N

This should be usable as a quality metric and CI artifact.

28. Human review improves the rules

Manual review must feed improvements back into the deterministic pipeline.

If hundreds of records fail for the same structural reason:

```
do NOT manually fix hundreds of YAML files
```

Instead:

```
identify pattern
  ↓
add rule/exception
  ↓
rerun
  ↓
resolve entire class
```

Generated YAML should generally not contain hand-authored fixes that cannot be reproduced.

A singleton may become a one-record exception, but it still belongs in the exception registry.

29. Suggested code organization

A reasonable structure is:

```
src/
  sources/
    census_governments.py
    census_tiger.py
    ocd_master.py

  normalize/
    government.py
    geography.py
    names.py

  resolution/
    government_division.py

  ocddid/
```

```
parser.py
builder.py
rules.py
exceptions.py
matcher.py

review/
  quarantine.py
  suggestions.py

models/
  division.py
  jurisdiction.py

render/
  division_yaml.py
  jurisdiction_yaml.py

graph/
  loader.py

config/
  ocdd_rules.yaml
  ocdd_exceptions.yaml
  tiger_layers.yaml

data/
  raw/
  cache/
  quarantine/

divisions/
jurisdictions/
```

Reuse existing OpenStates/Open Civic Data modules where they already implement these concerns.

Do not rewrite useful canonical parsing or exception behavior simply for architectural purity.

30. CLI behavior

Make pipeline stages independently rerunnable.

Conceptual commands:

```
jurisdictions sources census-governments --year 2025

jurisdictions sources tiger --year 2025

jurisdictions sources ocd-master

jurisdictions resolve --state CA

jurisdictions review export --state CA

jurisdictions generate --state CA

jurisdictions validate --state CA

jurisdictions graph build
```

Development runs should support filters:

```
jurisdictions resolve
  --state CA
  --government-type municipality
  --limit 100
```

Do not require network calls after source caches have been created unless explicitly refreshing external sources.

31. Recommended implementation order

Do not attempt all U.S. governmental structures simultaneously.

Start with a representative state such as California.

Implement deterministic adapters first for:

```
State government
County government
Municipal government
Town/township government where applicable
School districts
```

Each resolver should return either:

```
Resolved(...)
```

or a structured unresolved result:

```
Unresolved(  
    status=...,  
    reason=...  
)
```

Once those high-confidence cases work, run nationally.

Treat special districts as a separate phase.

32. Definition of a successfully generated Division

A normal TIGER-backed Division should ultimately contain, as appropriate:

```
stable UUID  
canonical OCD Division ID  
authoritative name  
display name  
classification  
external Census identifiers  
source/provenance  
temporal validity  
one or more temporal geometry references  
direct GeoJSON geometry URL  
relationships where needed
```

Do not require local geometry storage.

33. Definition of a successfully generated Jurisdiction

A Jurisdiction should ultimately contain, as appropriate:

```
stable UUID  
canonical Jurisdiction OCDID
```

```
official government name
display name
classification
Census government-unit identifier
relationship to one or more Divisions
official URLs when known
source/provenance
temporal validity
```

Unknown enrichment fields should remain nullable rather than preventing core record creation.

34. Source-of-truth hierarchy

Use approximately this authority order:

```
Government existence
  Census Government Organization / GUS

Government ID
  Census Government Organization

Government official name/type
  Census Government Organization

Division existence
  Census TIGER where applicable

Census geographic identity
  GEOID / relevant TIGER identifiers

Division boundary
  TIGERweb or another authoritative government geometry source

OCD Division identity
  canonical Open Civic Data master repository

OCD path generation
  existing IDs → existing OCD conventions → deterministic rules → exceptions

Website
  Census → .gov registry → authoritative candidate resolver → search fallback
```

35. Key conceptual constraints

Never equate:

government headquarters location

with:

government jurisdiction

Never equate:

Census government unit

with:

TIGER polygon

without a validated mapping.

Never create a fake GEOID for a special district.

Never automatically establish a new OCDID because a generated path looks reasonable.

Never make graph-database state authoritative over YAML.

Never use Git timestamps as substitutes for real-world validity dates.

Never overwrite stable entity identity because mutable attributes changed.

Never require a GIS database merely to return a Division's geometry.

36. Desired end state

The repository should become an open, reproducible national civic graph source.

A consumer should be able to take only the repository and derive relationships such as:

```
United States
  CONTAINS
California
  CONTAINS
Alameda County
  CONTAINS
Berkeley
```

and:

```
City of Berkeley
  GOVERNS
Berkeley
```

and:

```
Berkeley
  IDENTIFIED_BY
Census GEOID
```

and:

```
Berkeley
  HAS_GEOMETRY
TIGERweb GeoJSON resource
```

while also retaining historical facts such as:

```
this boundary applied from date A through date B
```

and provenance such as:

```
this geometry came from Census TIGER release X
```

The final system should favor:

```
authoritative sources
+ deterministic resolution
+ canonical OCD identity
```

- + explicit exceptions
- + review quarantine
- + temporal facts
- + reproducible YAML

over:

- search-heavy discovery
- + opaque fuzzy matching
- + hand-edited output
- + GIS infrastructure requirements
- + irreversible generated IDs

The goal is not merely to generate 98,000 YAML files once.

The goal is to create a maintainable pipeline that can repeatedly regenerate and update a comprehensive representation of U.S. governments and their geopolitical divisions as governments, boundaries, source datasets, and redistricting plans change over time.